

Liquidity Party

A Multi-Asset Logarithmic Market Scoring Rule AMM Deployment Whitepaper

Liquidity Party

Draft — [STUB: Publication date]

Abstract

Liquidity Party is a permissioned-deploy, non-upgradeable multi-asset automated market maker built for liquidity providers. Every rounding, approximation, and capacity decision in the system is biased strictly in favor of LPs: the swap kernel never pays a trader more than the true liquidity-sensitive LMSR mapping would, every $Q_{64.64}$ primitive in the single-asset chains is wrapped with ceiling helpers that restore LP-favor where ABDK’s default floor would leak a sub-ulp gift to the swapper, and every fee/protocol-share rounding decision rounds up for the pool and down for the protocol. The pricing kernel is the Logarithmic Market Scoring Rule (LMSR) with the liquidity parameter tied to pool size as $b(\mathbf{q}) = \kappa S(\mathbf{q})$, giving a scale-invariant pool whose maximum LP loss is bounded by $\kappa S(\mathbf{q}) \ln n$. Swaps execute against a two-pass midpoint- b approximation of the path-exact LS-LMSR cost function with a one-sided LP-safety guarantee (§3.4); single-asset deposits (`swapMint`) and single-asset withdrawals (`burnSwap`) replicate the externally observable price path by composing the fee-free kernel against a mutating local state, eliminating the round-trip drift that frozen- b Hanson incurs. Pool parameters are immutable post-deployment; the only mutable admin lever on a live pool is the protocol-fee recipient and the irreversible `kill()` switch. The admin cannot mint LP, change fees, freeze users, or move pool reserves — a leaked admin key presents little to no danger to LPs (§2.3). Pool creation is restricted to the trusted operator, who is responsible for upstream token vetting via a documented validator pipeline. This document specifies the pricing kernel and liquidity operations, the production fee structure, the admin and deployment surface, and the security-review methodology applied prior to launch, all framed for an LP audience.

Contents

1	Overview	3
2	LP Protection Mechanisms	4
2.1	Kernel-Level LP Safety	4
2.2	Fee and Protocol-Share Rounding	4
2.3	Admin-Side LP Protection	5
2.4	Burn Is Always Live	5
3	Pricing Kernel	5
3.1	LMSR Cost Function and the $\kappa S(\mathbf{q})$ Parameterization	5
3.2	Marginal Prices	6
3.3	Swap Mapping: Two-Pass Midpoint- b	6
3.4	LP-Safety Property of Midpoint- b	7
3.5	Slippage Protection	8

4	Liquidity Operations	8
4.1	Proportional Mint and Burn	8
4.2	Single-Asset Mint: <code>swapMint</code>	8
4.3	Single-Asset Redeem: <code>burnSwap</code>	9
5	Fee Structure	10
5.1	Per-Asset Fees	10
5.2	Single-Asset Fees on <code>swapMint</code> and <code>burnSwap</code>	10
5.3	Protocol Fees	10
5.4	Flash Loans	11
5.5	Rounding Policy	11
6	Funding Modes	11
7	Contract Architecture	12
7.1	PartyPool: Proxy and Storage	12
7.2	PartyPlanner: Factory and Registry	12
7.3	PartyInfo: View Helpers	12
7.4	PartyConcierge: Router	13
8	Admin Powers and Parameter Fixity	13
8.1	Mutable State on a Live Pool	13
8.2	Emergency Kill Switch	13
8.3	What the Operator Cannot Do	14
9	Deployment Guidelines	14
9.1	Operator Workflow	14
9.2	Choosing κ	14
9.3	Choosing Bases β_i	15
9.4	Fee Calibration	15
9.5	Native-Asset Wrapping	16
10	Security Review Process	16
10.1	Layered Defence	16
10.2	Internal Review Artefacts	16
10.3	Tooling	17
10.4	External Audit	17
10.5	Bug Bounty	18
10.6	Vulnerability Disclosure	18
10.7	Out of Scope	18
11	Risk Management	18
11.1	Slot Drainage Is Expected LP Behaviour	18
11.2	LP Loss Bound	19
11.3	Liquidity-Manipulation Resistance	19
11.4	Numerical Envelope	19
11.5	Operational Mitigations	20
11.6	Out-of-Scope Token Classes	20

12 Deployment Surface	20
13 Disclaimers	21
References	21

1 Overview

Liquidity Party is a multi-asset AMM that supports up to $n \leq 383$ tokens in a single pool and quotes every pair within that pool from a single convex potential. The system is designed for capital efficiency on long-tail asset baskets: an n -asset pool exposes $n(n-1)/2$ pairs with one set of LP shares, eliminating the fragmentation cost of pairwise CFMM deployments. Approximate gas costs for production-typical operations are summarized in Table 1.

Table 1: Approximate gas costs by pool size

Assets	Pairs	Swap	Mint	swapMint
2	1	127,000	144,000	150,000
10	45	139,000	412,000	370,000
20	190	154,000	746,000	694,000
50	1225	199,000	1,751,000	2,000,000

System layout. The on-chain system consists of four singleton-or-immutable contracts:

- **PartyPlanner** — factory and pool registry. `onlyOwner newPool` deploys each **PartyPool** via `CREATE2` with caller-specified $(\kappa, f, f_{\text{flash}})$. Indexes pools by token for off-chain discovery.
- **PartyPool** — the pool itself, holding token reserves and issuing an ERC-20 LP share token. The pool is a non-upgradeable `DELEGATECALL` proxy over two implementation libraries (`PartyPoolMintImpl`, `PartyPoolExtraImpl`) due to the EIP-170 24 KB contract-size cap.
- **PartyInfo** — read-only view contract for prices, quotes, and denominators. Pricing helpers are exposed here rather than on the pool so that **PartyPool** fits its size budget.
- **PartyConcierge** — user-facing router. Takes token addresses rather than indices, supports a single ERC-20 approval covering every pool, and exposes an EIP-7730 clear-signing surface. Funds pools via the callback funding mode (see §6).

The pool’s per-token configuration (bases and per-asset fee rates) is stored in an `SSTORE2`-style data contract referenced by an immutable address in the pool. Hot paths read it via `EXTCODECOPY`; no `SLOAD` on the bases or fees occurs on the swap path.

Notation. We index assets by $i \in \{0, \dots, n-1\}$ and write the pool’s internal inventory as $\mathbf{q} = (q_0, \dots, q_{n-1})$, with the size metric $S(\mathbf{q}) = \sum_i q_i$. The liquidity parameter is $b(\mathbf{q}) = \kappa S(\mathbf{q})$ for a positive constant κ fixed at deployment. Internal quantities are maintained in Q64.64 fixed-point (ABDKMath64x64); user-facing token amounts are integer base units (`uint256`) and are converted on the boundary using per-asset denominators β_i (the *bases*) set at deployment: $q_i = \text{balance}_i / \beta_i$.

Primary audience: liquidity providers. Liquidity Party’s first concern is the LP. The protocol is engineered so that every decision the LP cannot make on their own behalf — rounding, approximation, cap-and-invert branch selection, fee/protocol-share split, emergency response — is biased in favor of the LP. Section 2 consolidates the explicit LP-favor guarantees; sections 3–§5 reference them at each construction step. Takers and integrators are first-class users of the protocol surface but inherit the LP-favor invariants from the LP side; they should design assuming that any sub-ulp ambiguity in the rounding chain will resolve in the LP’s favor and against their own position.

Reading guide. §2 catalogues LP protection mechanisms. §3–§4 specify the pricing kernel and liquidity operations as implemented. §5–§6 describe fees, flash loans, and the four funding modes. §7 covers the deployed contract surface, §8 the admin powers and parameter fixity, and §9 the operator’s deployment-time calibration choices. §10 documents the security review process that gates production launch. §11 states the residual risk surface and operational mitigations, including the framing of slot drainage as expected and industry-normal LP behaviour.

2 LP Protection Mechanisms

This section consolidates the LP-favor guarantees that the rest of the document references. Each mechanism is enforced in source; this is an index, not a statement of intent.

2.1 Kernel-Level LP Safety

Midpoint- b never overshoots LS-LMSR. The production swap kernel (`LMSRStabilized.swapAmountsForExact`) is a two-pass midpoint approximation of the path-exact LS-LMSR mapping (§3.4). The output it pays the swapper is bounded above by the output the path-exact LS-LMSR mapping would pay under the same inputs — the swapper *never* receives more than the path-exact reference, and the LP *never* loses more than the path-exact reference. The bound is empirically verified on a 26-point grid covering $N \in \{2, 5, 10, 50\}$, $\kappa \in \{0.005, \dots, 2.0\}$, $a/q \in \{10^{-4}, \dots, 10^{-1}\}$, plus a 3-point asymmetric grid, to 10^{-10} absolute tolerance.

Capacity caps. Output is hard-capped at available inventory $y \leq q_j$ in the kernel (§3.3). The cap-and-invert branch derives the input that the LP actually absorbs, so the pool is never required to over-pay relative to its inventory.

Single-asset chain LP-favor rounding. The `swapMint` and `burnSwap` chains (§4.2–§4.3) carry a per-primitive rounding discipline. Where ABDK’s default `mul/div/exp/ln` floor would shave a sub-ulp gift to the swapper, the kernel substitutes a ceiling helper (`_ceilMul`, `_ceilDiv`, `_ceilExp`, `_ceilLn`); where the floor direction already serves LP-favor, the default is kept. Every individual decision is documented in-source by signed partial derivative against the LP-favoring direction. The `EXP_LIMIT` branch in `swapMint` carries a 1-ulp cushion on top of the ceiling chain so that future ABDK-level rounding changes cannot re-open a leak.

2.2 Fee and Protocol-Share Rounding

- **Total fees round up** for the pool and against the taker.
- **Protocol share rounds down** for the LP and against the protocol developers.

- **Outputs to users floor; inputs from users ceil.**
- **Pre-funded dust** is credited to LP reserves rather than refunded to the caller.

The protocol developers absorb the rounding loss on the protocol-share split; the LP keeps the residual. This is deliberate.

2.3 Admin-Side LP Protection

The admin (planner owner / pool owner) cannot reach LP reserves. The exhaustive CAN/CANNOT inventory is in `doc/security/admin-powers.md`; the LP-relevant summary is:

- Admin **cannot** mint LP out of band, burn user LP, change κ , change fees on a live pool, pause user-side **burn**, blacklist or freeze user balances, upgrade the pool implementation, renounce ownership, or move funds out of the pool.
- Admin **can** deploy new pools, set the protocol-fee recipient on a live pool, and invoke the irreversible `kill()` switch on a pool they own.

Compromised-key worst case. A leaked admin key cannot reach LP reserves. Worst case: the attacker calls `kill()` (disabling new deposits, swaps, flash loans, single-asset withdrawals, and protocol-fee collection) and rotates `protocolFeeAddress` so any not-yet-collected protocol fees route to the attacker. **burn** remains operational on a killed pool, so LPs can always withdraw their pro-rata share of pool reserves. The core team retains the ability to `kill()` the system as a final defensive measure — even if the admin key is compromised, the system can be put into the LP-can-only-exit state. **A leaked admin key presents little to no danger to LP reserves;** it is a denial-of-service surface and a partial protocol-fee redirect surface, not a fund-loss surface.

The admin key is held on a hardware wallet (see §12).

2.4 Burn Is Always Live

burn (the proportional-redemption path) is the only state-mutating entry point that `kill()` does *not* disable. The proportional-redemption guarantee is preserved across every kill scenario, every admin-key scenario, and every operator-misconfiguration scenario by construction: there is no code path in the protocol that can disable **burn** for an LP holding the LP token.

3 Pricing Kernel

3.1 LMSR Cost Function and the $\kappa S(\mathbf{q})$ Parameterization

The pool maintains the LMSR cost function in *inventory convention*

$$C(\mathbf{q}) = -b(\mathbf{q}) \ln \left(\sum_{k=0}^{n-1} e^{-q_k/b(\mathbf{q})} \right), \quad b(\mathbf{q}) = \kappa S(\mathbf{q}), \quad (1)$$

where larger q_i corresponds to a more abundant slot and therefore a *cheaper* marginal price. The proportional parameterization $b(\mathbf{q}) = \kappa S(\mathbf{q})$ yields three properties that drive the rest of the design:

1. **Scale invariance.** A proportional rescaling $\mathbf{q} \mapsto \lambda \mathbf{q}$ scales b by λ and leaves every pairwise marginal price unchanged, so proportional mint and burn are value-neutral.

2. **Bounded LP loss.** The classical LMSR worst-case-loss $b \ln n$ becomes $\kappa S(\mathbf{q}) \ln n$ at the current state; per unit of size this is $\kappa \ln n$, fixed by the deployment parameters.
3. **Liquidity sensitivity.** As the pool absorbs trades and accrues fees, $S(\mathbf{q})$ grows and b grows with it; depth deepens automatically in proportion to TVL without governance intervention.

The continuous LS-LMSR construction of Othman et al. (2013) admits a similar size-dependent b but with a continuously evolving liquidity trajectory whose exact pricing requires integral evaluation. Our $b(\mathbf{q}) = \kappa S(\mathbf{q})$ choice is a fixed proportionality whose effect we recover via a two-pass numerical midpoint (§3.3) rather than a path integral.

3.2 Marginal Prices

With b treated quasi-statically, the gradient of C recovers softmax-style prices and the instantaneous marginal exchange rate for swapping input $i \rightarrow$ output j is

$$P(i \rightarrow j \mid \mathbf{q}) = \exp\left(\frac{q_j - q_i}{b(\mathbf{q})}\right). \quad (2)$$

A more abundant output slot (larger q_j) buys more output tokens per unit input. Per-pair denomination-adjusted prices are exposed via `PartyInfo.price(pool, i, j)` as a Q128.128 fixed-point quantity that accounts for the bases β_i, β_j .

3.3 Swap Mapping: Two-Pass Midpoint- b

Let i, j be the input and output asset indices and let a be the fee-discounted internal input. The production swap kernel (`LMSRStabilized.swapAmountsForExactInput`) computes the output y via a two-pass Heun-style midpoint of the Hanson exact-input formula:

Pass 0 (frozen pre-state b):

$$\begin{aligned} b_0 &= \kappa S(\mathbf{q}), \quad r_0 = \exp((q_j - q_i)/b_0), \\ y_0 &= b_0 \ln(1 + r_0 (1 - e^{-a/b_0})), \end{aligned} \quad (3)$$

Pass 1 (midpoint b):

$$\begin{aligned} b_{\text{mid}} &= \kappa \left(S(\mathbf{q}) + \frac{a - y_0}{2} \right), \quad r_{\text{mid}} = \exp((q_j - q_i)/b_{\text{mid}}), \\ y &= b_{\text{mid}} \ln(1 + r_{\text{mid}} (1 - e^{-a/b_{\text{mid}}})) \end{aligned} \quad (4)$$

The first pass freezes b at the pre-swap size, producing a Hanson output that under-estimates the true liquidity-sensitive LS-LMSR fill (because b grows as the trade absorbs input). The second pass re-evaluates the Hanson mapping at the size metric averaged across the swap, providing a Heun-style second-order correction. The implementation caps output at the available inventory q_j when the inner term of $\ln$ degenerates (cap-and-invert branch).

Why two passes. A single-pass Hanson evaluation at frozen pre-state b exhibits a round-trip asymmetry between `swapMint` and `burnSwap` on the order of ~ 200 bps in worst-case adversarial regimes. The midpoint- b kernel reduces this asymmetry to ≤ 2 bps, which is covered by per-swap fees in any closed cycle.

3.4 LP-Safety Property of Midpoint- b

The path-exact reference is the liquidity-sensitive LMSR mapping under inventory convention: the y^* that solves the cost-preservation equation

$$C(\mathbf{q} + a e_i - y^* e_j) = C(\mathbf{q}), \quad C(\mathbf{q}) = -b(\mathbf{q}) \ln\left(\sum_k e^{-q_k/b(\mathbf{q})}\right), \quad b(\mathbf{q}) = \kappa S(\mathbf{q}). \quad (5)$$

With b state-dependent, both b and the inner exponents change as the state moves from $\mathbf{q}$ to $\mathbf{q} + a e_i - y e_j$, so y^* has no closed form.

LP-safety theorem (empirically verified). *For every $(a, \mathbf{q}, \kappa, n)$ in the production envelope, the midpoint- b output y_{mid} satisfies*

$$y_{\text{mid}} \leq y^*.$$

Equivalently, the swapper receives at most what the path-exact LS-LMSR would pay, and the LP loses at most what the path-exact LS-LMSR would surrender.

Structural argument. The starting point is the frozen-pre-state Hanson output y_0 , which satisfies $y_0 < y^*$: with b growing along the deposit path ($S(\mathbf{q})$ rises from $S(\mathbf{q})$ to $S(\mathbf{q}) + (a - y^*)$), the cost surface flattens and the cost-preserving y at the path-exact b -trajectory exceeds the y at the frozen b . The midpoint correction replaces $b_0 = \kappa S(\mathbf{q})$ with

$$b_{\text{mid}} = \kappa \left(S(\mathbf{q}) + \frac{a - y_0}{2} \right),$$

which is the size metric at the midpoint of the *frozen- b* swap trajectory. Because $y_0 < y^*$ implies

$$S(\mathbf{q}) + \frac{a - y_0}{2} < S(\mathbf{q}) + \frac{a - y^*}{2},$$

b_{mid} is strictly below the size metric at the midpoint of the *true* swap trajectory. The Hanson exact-in mapping is increasing in b on the relevant regime, so the recomputed y_{mid} is less than the path-exact midpoint-rule output, which itself is bounded above by y^* (the true cost-preserving fill). Both bounds operate in the LP-favouring direction.

Empirical verification. The property is verified by exhaustive grid testing in `test/MidpointBSwapGas.t.sol` (`test_midpoint_NeverOvershoots_LSLMSR`, `test_midpoint_NeverOvershoots_AsymmetricPool`). The grid covers $N \in \{2, 5, 10, 50\}$, $\kappa \in \{0.005, 0.05, 0.2, 0.5, 1.0, 2.0\}$, and $a/q \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$, on both symmetric and asymmetric starting states. Tolerance: 10^{-10} absolute, well below 1 wei in Q64.64. **No grid point overshoots.** The bound is therefore established by a combination of the structural argument above and exhaustive testing across the production envelope, rather than by a closed-form proof over the unbounded $(a, \mathbf{q}, \kappa, n)$ space.

Consequence. A `swapMint/swapBurn` closed cycle pays at most the path-exact LS-LMSR round-trip cost plus the residual midpoint- b approximation drift (≤ 2 bps, see `security/drainage-guidelines.md` §7). Per-swap fees uniformly exceed this drift at any deployment-recommended fee level, so closed cycles are strictly loss-making for the attacker and strictly profitable for the LP.

Capacity caps. For output y that would exceed available q_j , the kernel returns $y = q_j$ at the cap-and-invert boundary. The inverse mapping (`amountInForExactOutput`) returns the input required for a target output, using

$$a(y) = b \ln\left(\frac{r_0}{r_0 + 1 - e^{y/b}}\right), \quad (6)$$

which is the exact closed-form inverse of the Hanson exact-in mapping (used internally, not as a public exact-out swap entry point).

Numerical stability. The kernel computes ratios like $r_0 = \exp((q_j - q_i)/b)$ directly rather than dividing exponentials, guards every `exp` and `ln` call to bounded domains (`EXP_LIMIT = 32.0`), and checks the positivity of the inner `ln` argument before evaluation. Domain violations revert; outputs are clamped to capacity rather than extrapolated. The cost function evaluates log-sum-exp with one-pass dynamic recentering for stability under dispersed q_k .

3.5 Slippage Protection

Every user-facing swap accepts a `minAmountOut` (or, on `swapMint`, a `maxAmountIn`). If the computed output is below `minAmountOut`, the call reverts. This is denomination-aware slippage protection requiring no price-unit conversions. Off-chain callers that prefer a price-based bound may use the bisection helper `PartyInfo.swapAmountsForExactPrice` to convert a target post-swap forward price into a `(maxAmountIn, minAmountOut, fee)` triple suitable for direct submission to `swap()`.

4 Liquidity Operations

The pool issues an ERC-20 LP share token. The total LP supply L tracks $S(\mathbf{q})$ proportionally: $L^{(0)} = 10^{18}$ shares are minted at `initialMint` regardless of pool size, and subsequent operations preserve the ratio $L/S(\mathbf{q})$ on every proportional change.

4.1 Proportional Mint and Burn

mint(lpTokenAmount). The caller specifies a target LP amount ΔL . The pool computes the required deposit amounts as the proportional fraction $\Delta L/L$ of each current reserve, pulls those amounts via the chosen funding mode (§6), and mints exactly ΔL shares to `receiver`. Any over-delivery from a funding callback is absorbed into reserves (donated to LPs) per the rounding policy (§5.5). LP shares minted are derived from the post-deposit size delta divided by the pre-deposit cached size to prevent fee skim during the first mint after fee accrual.

burn(lpAmount). The caller burns ΔL shares and receives the proportional fraction $\Delta L/L$ of each reserve token. `burn` is the *only* state-mutating entry point that is *not* disabled by `kill()` (see §8); LPs can always exit pro rata.

Proportional mint and burn are value-neutral in the fee-free kernel: every pairwise marginal price is invariant under $\mathbf{q} \mapsto \lambda \mathbf{q}$.

4.2 Single-Asset Mint: `swapMint`

`swapMint` is exact-LP-out: the caller specifies ΔL shares to receive and a `maxAmountIn` cap on the deposit of a single input asset i . Internally, the operation is equivalent to executing the externally observable sequence “swap $i \rightarrow j$ for every $j \neq i$, then proportional mint with the resulting basket.”

The fee-free kernel computes the required input in one forward pass against a memory-only local state $\mathbf{q}_{\text{local}}$:

1. Compute $\gamma = \Delta L / L$ and $\beta = \gamma / (1 + \gamma) \in (0, 1)$.
2. Initialize $\mathbf{q}_{\text{local}} = \mathbf{q}$.
3. For each $j \neq i$ in fixed index order: recompute $b_{\text{local}} = \kappa S(\mathbf{q})[\mathbf{q}_{\text{local}}]$, $r_{0,j} = \exp((q_{\text{local}}^j - q_{\text{local}}^i) / b_{\text{local}})$, and solve the Hanson exact-out inverse for the cost of acquiring βq_j of asset j :

$$x_j = b_{\text{local}} \ln \left(\frac{r_{0,j}}{r_{0,j} + 1 - e^{\beta q_j / b_{\text{local}}}} \right).$$

Update $\mathbf{q}_{\text{local}}$: $q_{\text{local}}^i += x_j$, $q_{\text{local}}^j -= \beta q_j$.

4. Return the total deposit $a = \frac{\beta q_i + \sum_{j \neq i} x_j}{1 - \beta}$.

The chain mutates a memory copy only; the pool's storage is updated at the end with the new reserves and exactly ΔL LP shares are minted. Each per-leg call re-evaluates b_{local} at the current chain state, so the simulation matches the post-state that any external actor would produce by performing the same sequence of swaps. As a consequence, a **swapMint-burnSwap** round trip incurs only composed swap slippage rather than the frozen- b drift produced by a single-shot kernel evaluation.

Rounding. The exact-out cost x_j uses ABDK floor primitives that round in the trader's favor under the natural **mul/div/exp/ln** semantics. To restore the LP-favoring invariant, the kernel applies explicit ceiling helpers (**_ceilMul**, **_ceilDiv**, **_ceilExp**, **_ceilLn**) at every step whose floor direction would shave a sub-ulp gift to the swapper, and adds a 1-ulp cushion in the asymptotic **EXP_LIMIT** branch. Every individual rounding decision is documented in-source by signed partial derivative against the LP-favoring direction.

4.3 Single-Asset Redeem: **burnSwap**

burnSwap is the inverse of **swapMint**: the caller burns ΔL LP shares and receives a single output asset j . The fee-free kernel computes the payout by simulating a proportional withdraw followed by swaps of every withdrawn a_k back into asset j against the already-burnt local state:

1. Compute $\alpha = \Delta L / L$.
2. Initialize $\mathbf{q}_{\text{local}} = (1 - \alpha) \mathbf{q}$ and start the payout with the proportional share $Y_j = \alpha q_j$.
3. For each $k \neq j$: set $a_k = \alpha q_k$ (the withdrawn portion of asset k). Recompute b_{local} and $r_{0,k} = \exp((q_{\text{local}}^j - q_{\text{local}}^k) / b_{\text{local}})$ at the current state; compute the Hanson exact-in payout

$$y_{k \rightarrow j} = b_{\text{local}} \ln \left(1 + r_{0,k} (1 - e^{-a_k / b_{\text{local}}}) \right).$$

Accumulate $Y_j += y_{k \rightarrow j}$ and update $\mathbf{q}_{\text{local}}$. If $y_{k \rightarrow j}$ exceeds available q_{local}^j , cap the payout to q_{local}^j and derive the actually-consumed input via the exact-out inverse.

4. Per-asset numerical degeneracy (e.g. $b_{\text{local}} \leq 0$, non-positive inner $\ln$ argument) is treated as zero contribution from that asset rather than aborting the redeem.

The cap-hit branch uses ceiling rounding for the simulated input absorbed by the pool (so subsequent legs see a larger q_{local}^k and produce strictly smaller payouts), and shaves one ulp from the cap payout, preserving LP favor without exposing the burner to a denial-of-service via per-leg failure.

Kill-resistance asymmetry. `burnSwap` is disabled by `kill()`; `burn` is not. A killed pool therefore permits proportional redemption but not single-asset redemption.

5 Fee Structure

5.1 Per-Asset Fees

Each asset k has an immutable fee rate f_k set at deployment, expressed in parts-per-million. The deployer enforces $f_k < 10,000$ ppm ($< 1\%$). For an asset-to-asset swap $i \rightarrow j$, the effective pair fee is the additive composition

$$f_{\text{eff}}(i, j) = f_i + f_j, \quad (7)$$

applied to the user’s submitted input before the fee-free kernel:

$$a_{\text{eff}} = a_{\text{user}}(1 - f_{\text{eff}}).$$

The additive form is preferred over the exact multiplicative $1 - (1 - f_i)(1 - f_j)$ because the cross-term at the deployment cap ($f_i, f_j < 10^{-2}$) is below 10^{-4} of the pair fee, and the simpler formula is cheaper to evaluate and easier for users to reason about.

The collected fee is retained in the input token and split into a protocol share and an LP share (see below). The LP share remains in pool reserves, raising $S(\mathbf{q})$ and thereby the LP-share-denominated value.

5.2 Single-Asset Fees on `swapMint` and `burnSwap`

Single-asset operations interact with exactly one external token, so only that token’s fee applies:

- **swapMint:** charge f_i on the deposited token before invoking the fee-free kernel. The protocol share is split off; the remainder increases pool reserves.
- **burnSwap:** charge f_j on the gross payout after the fee-free kernel computes it. The protocol share is split off; the remainder stays in pool reserves.

There is no separate LP-token-side fee. This keeps LP accounting symmetric with the asset-to-asset path and avoids double-charging when LP shares are the counterparty.

5.3 Protocol Fees

A protocol-fee share ϕ (ppm) is taken from every collected fee. The deployer enforces $\phi < 400,000$ ppm ($< 40\%$). Accrued protocol fees are tracked in a per-token ledger (`_protocolFeesOwed`) that is excluded from the pool’s liquidity-bearing balance (`_cachedUintBalances`); pending protocol fees do not participate in pricing or earn further fees. The `collectProtocolFees()` entry

point sweeps the ledger to the configured `protocolFeeAddress` and zeroes it; the recipient is read at call time, so a recipient rotation applied between accrual and collection retroactively redirects pending fees (Uniswap V3-style semantics; operators that need atomic handoff must collect before rotating).

5.4 Flash Loans

The pool implements ERC-3156. Any caller may borrow any pool token within a single transaction, repaying principal plus the flash-loan fee f_{flash} (ppm, immutable, capped at $< 10,000$ ppm) before the borrower callback returns. The protocol share ϕ of the flash fee is accumulated to `_protocolFeesOwed`; the remainder remains in reserves and accrues to LPs analogously to swap fees. Flash loans are disabled by `kill()`.

5.5 Rounding Policy

All rounding decisions are biased to protect existing LP value:

- Total fees are rounded *up* (taker pays the extra ulp; LP keeps it).
- The protocol share is rounded *down* (LP keeps the residual; the protocol developers absorb the ulp).
- Pool outputs to users are floored; pool inputs from users are ceiled.
- Pre-funded amounts above the exact required amount are credited to reserves rather than refunded (LP gift).
- Q64.64 ceil helpers (`_ceilMul`, `_ceilDiv`, `_ceilExp`, `_ceilLn`) are applied per-primitive in `swapMint`/`burnSwap` to restore LP favor where ABDK's default floor would otherwise leak a sub-ulp gift to the swapper.

6 Funding Modes

User-facing entry points (`swap`, `mint`, `swapMint`, `burnSwap`) accept any of four token-delivery modes via a `fundingSelector` parameter. The kernel mathematics are identical across modes.

APPROVAL Standard ERC-20 `transferFrom` from the caller. Caller must hold the input balance and have approved the pool. The mode requires `msg.sender == payer` to prevent third-party-allowance abuse.

PREFUNDING Caller deposits tokens to the pool address before invoking the entry point; the pool measures the balance delta against `_cachedUintBalances + _protocolFeesOwed`. Excess above the required amount is gifted to LPs. Native ETH is auto-wrapped via the configured wrapper when the pool's input token equals the wrapper address and the pool's ETH balance covers the amount.

CALLBACK Aggregators and routers implement a callback interface; the pool calls `payer.fundingSelector(nonce, token, amount, cbData)` and measures the resulting balance delta to determine the funded amount.

PERMIT2 Uniswap Permit2 signature-based transfer with witness data binding the operation’s parameters to the signature. Witness types are defined per entry point in `PartyPoolPermit2Witness` for `swap`, `mint`, and `swapMint`; `burnSwap` does not require an input transfer.

The flash-loan entry point does not use a funding mode: principal is lent out and repayment is pulled via `transferFrom` after the borrower callback returns.

Native ETH. The pool accepts native ETH on the entry points declared `payable`, but only the configured wrapper (a WETH-style contract) may push raw ETH back into the pool via `receive()`. Any other direct ETH transfer reverts so stray sends cannot be stranded. Multi-asset mint loops carry an explicit per-call ETH budget so the wrap branch cannot over-claim against a constant `msg.value`.

7 Contract Architecture

7.1 PartyPool: Proxy and Storage

`PartyPool` is a non-upgradeable `DELEGATECALL` proxy. Two implementation libraries (`PartyPoolMintImpl`, `PartyPoolExtraImpl`) host the body of `mint`, `burn`, `swapMint`, `burnSwap`, `flashLoan`, `collectProtocolFees`, and the constructor body. The pool’s own runtime is a thin facade so its deployed bytecode plus its creation-code embed in `PartyPoolInitCode` both fit under EIP-170’s 24 KB contract-size cap. Implementation addresses are immutable; there is no upgrade path.

Storage layout is mirrored by `PartyPoolStorage.PoolState` for use under `DELEGATECALL`. Per-token `bases` and `fees` are not held in storage — they live in an `SSTORE2` “BFStore” data contract deployed at `init` time, whose address is captured in the `IMMUTABLE_BFSTORE` pool immutable. Hot paths read it via `EXTCODECOPY`, avoiding bounds-check `SLOADs` on the array length.

The LMSR state (`_lmsr.kappa`, `_lmsr.qInternal`) is a cache: the pool re-derives the post-swap, post-mint, and post-burn `qInternal` from the cached uint balances and the immutable bases on every mutation, keeping the kernel state synchronized with physical reserves across all state-changing paths.

7.2 PartyPlanner: Factory and Registry

`PartyPlanner.newPool` (`onlyOwner`) deploys a new `PartyPool` via `CREATE2` with operator-supplied parameters (see §8). The planner indexes deployed pools and tokens for off-chain discovery (`getAllPools`, `getPoolsByToken`, `tokenIndex`). A delta-equality check on the initial deposit (`balanceAfter - balanceBefore == initialDeposits[i]`) at deploy time rejects fee-on-transfer and rebasing tokens that fail to deliver the exact deposit amount. Pre-existing balances at the `CREATE2` address are absorbed into reserves and gifted to the first depositor; this is intentional and closes a deployment-griefing DoS that total-equality would have re-opened (see `doc/security/threat-model.md` §J.6).

7.3 PartyInfo: View Helpers

`PartyInfo` is a singleton view contract that accepts a target pool address. It hosts pricing utilities (`price`, `poolPrice`, `swapAmountsForExactPrice`, quote helpers for the multi-step mint/burn chains), the BFStore decoders (`denominators`, `fees`), and a `working(pool)` health check. Quote helpers run the same simulation as the on-chain entry points; their output is suitable for direct submission to `swap`/`swapMint`/`burnSwap`.

7.4 PartyConcierge: Router

PartyConcierge is the user-facing router. It accepts token addresses rather than indices so wallets can render trades with human-readable token names (EIP-7730 clear-signing compatibility). A single ERC-20 approval to the Concierge covers every pool on the same chain. Funding is callback-based: the Concierge stores call context in EIP-1153 transient storage (`cbUser`, `cbPool`, `cbMode`, `cbEthBudget`), validates `msg.sender == cbPool` on every callback, and refunds residual ETH via `sweepEth`. Adds approximately 10k gas to a typical swap.

The Concierge supports three input paths: standard approval pull, Permit2 signature-based pull (no Concierge approval required, only the universal Permit2 approval), and native-ETH wrap from `msg.value`.

8 Admin Powers and Parameter Fixity

The deployment parameter tuple

$$(\kappa, \mathbf{f} = (f_0, \dots, f_{n-1}), \phi, f_{\text{flash}}, \beta = (\beta_0, \dots, \beta_{n-1}))$$

is set at `newPool` and is immutable for the lifetime of the pool. The deployer enforces hard caps at construction time:

Table 2: Hard caps enforced at `PartyPlanner.newPool`

Parameter	Bound
Per-asset swap fee f_i	$< 10,000$ ppm ($< 1\%$)
Flash-loan fee f_{flash}	$< 10,000$ ppm ($< 1\%$)
Protocol fee share ϕ	$< 400,000$ ppm ($< 40\%$)
Asset count n	$2 \leq n \leq 383$
κ	> 0 (Q64.64)
Per-token base β_i	> 0

8.1 Mutable State on a Live Pool

The only mutable state on a deployed pool, post-`newPool`, is:

- `protocolFeeAddress` — recipient of accrued protocol fees, settable by the pool owner via `setProtocolFeeAddress`. Zero address is rejected when $\phi > 0$. The recipient is read at `collectProtocolFees` call time (see §5).
- Two-step ownership handoff via the `Ownable2Step` surface.
- The irreversible `kill()` flag.

8.2 Emergency Kill Switch

The pool owner may call `kill()`, after which all state-mutating entry points except `burn` revert: `swap`, `mint`, `swapMint`, `burnSwap`, `flashLoan`, `collectProtocolFees`, and `initialMint`. The action is one-way; there is no `unkill`. The proportional-redemption guarantee of `burn` is preserved by design so LPs can always withdraw their pro-rata share. `kill()` is intended for incident response

if a critical vulnerability is discovered post-deployment; it is disjoint from the immutable parameter set.

8.3 What the Operator Cannot Do

The operator (planner owner and pool owner) explicitly *cannot*: mint LP out of band, burn user LP, change κ or any fee parameter on a live pool, pause user-side burns, blacklist or seize user balances, upgrade the pool implementation, renounce ownership, or move funds out of the pool directly. The full inventory of admin powers (CAN and CANNOT) is documented in `doc/security/admin-powers.md`.

9 Deployment Guidelines

9.1 Operator Workflow

The recommended end-to-end deployment workflow is:

1. **Token vetting.** Run `bin/validate-token <addr>` against every candidate token (see `doc/security/trusted-deployer-policy.md` for the validator policy). Resolve every `WARN` off-chain; `FAIL` blocks listing. Cross-check with `slither-check-erc <addr> ERC20 -erc-conformance`. Record verifications in a listing record.
2. **Parameter selection.** Choose κ (§9.2), per-asset fees, flash fee, and bases (§9.3).
3. **Initial liquidity.** Fund the deployer with `initialDeposits[i]` for each token; approve the planner.
4. **Deploy.** Call `PartyPlanner.newPool(...)`. The planner deploys the pool via `CREATE2`, pulls initial liquidity from the operator-specified `payer`, and invokes `initialMint` on the new pool, minting 10^{18} LP shares to `receiver`.
5. **Verify.** Confirm the pool address, `LMSR()` state, and `balances()` match expectations. Subscribe to the `PartyStarted` event for the indexer.

9.2 Choosing κ

κ controls the trade-off between price stability (higher κ = more forgiving for large trades) and depth efficiency (lower κ = more sensitive, drains faster on adverse moves). The maximum external single-asset price move the pool can survive before that slot drains is

$$R_{p,\max} = \exp\left(\frac{1}{\kappa(n-1)}\right). \quad (8)$$

Production-recommended ranges, by pool type:

For multi-asset pools the same $\kappa S(\mathbf{q})$ liquidity budget is shared across n slots, so single-slot divergence drains faster as n grows. The canonical drain-threshold table is in `security/drainage-guidelines.md` §2.

Table 3: Suggested κ ranges by pool type (see `security/drainage-guidelines.md`)

Pool type	κ	Rationale
Tight stablecoin peg	0.5 – 2.0	Absorbs only small moves; toxic flow stays out
Correlated assets (stETH/ETH)	0.2 – 0.5	Tolerates modest decoupling
Volatile blue-chip pairs (ETH/BTC)	0.1 – 0.3	Designed for 2–20 $\times$ moves
Long-tail / speculative	0.01 – 0.1	Survives larger price discovery

9.3 Choosing Bases β_i

Each per-token base β_i converts raw token units to internal Q64.64 units: $q_i = \text{balance}_i / \beta_i$. Set β_i such that the initial q_i values lie within $\sim 10\times$ of each other across slots. This:

- Maximizes precision headroom across natural drift.
- Aligns the initial spot price near 1:1 in internal units; the deployer is responsible for any external-USD reference rate encoded in the initial deposit ratios.
- Sits the pool well inside the kernel’s EXP_LIMIT envelope.

Bases are immutable; revising them requires deploying a new pool. The bases are sourced from `initialDeposits` passed to `newPool`.

9.4 Fee Calibration

Per-asset fees use a $C \cdot \sigma$ **method**: the fee rate for asset i is set proportional to a volatility estimate σ_i for that asset, scaled by a single protocol-wide constant C . USDC anchors the table at 1.0 bps; every other listed asset’s fee is set so that f_i / σ_i matches USDC’s $f_{\text{USDC}} / \sigma_{\text{USDC}}$, then scaled by C .

Bridge surcharge. Wrapped-bridged assets (e.g. wSOL) carry an additional heuristically-applied fee component on top of the $C\sigma$ baseline. The surcharge reflects the bridge-trust risk premium: an LP holding a wrapped asset bears the bridge’s solvency and operational risk in addition to the underlying asset’s market risk, and the fee schedule compensates them for that.

Step 1: calibrate C to undercut competing pools. C is set as a first step, before any κ tuning, by reference to the fee levels charged by competing AMMs on the same or comparable pairs. C is raised to the largest value that still “just undercuts” the prevailing market rate on the relevant pairs — high enough to extract meaningful fee revenue from organic flow, low enough that fee-sensitive takers prefer Liquidity Party to the competing venue. Once chosen, C is fixed for the rest of the calibration.

Step 2: optimize κ at fixed C . With C fixed, κ is tuned against a 15-month historical simulation. The simulation operates two flow streams against a candidate pool:

1. A *Binance-connected arbitrageur* able to take infinite liquidity at the Binance inside quote plus 2 bps. This is an upper bound on real arbitrage activity — the inside is thin in practice, so a real arb cannot take infinite size at the inside plus 2 bps. The simulation therefore over-estimates arbitrage pressure on the pool, biasing κ toward conservative (LP-favoring) values.

2. *Organic flow* derived from on-chain trade data, specifically CoW Swap auction participant flow over the same 15-month window. This is real, fee-sensitive user demand.

The objective is the LP’s terminal net return: fees collected minus LVR-style adverse-selection cost minus realized IL on slot drift, measured at the simulated horizon. κ is chosen to maximize that net at the fixed C .

The procedure — C from market-rate undercut, then κ from LP-net-return optimization at fixed C — decouples the two parameters: C is set by competitive positioning and κ is set by historical LP-economics simulation. The same $C \cdot \sigma$ schedule and the same chosen κ are used across all initial pool deployments; future pools recalibrate C if the competitive landscape on the new pair differs materially, and re-run the κ optimization if the new flow profile is not covered by the existing simulation envelope.

9.5 Native-Asset Wrapping

If the pool will accept native ETH (or chain-native equivalent), one of the pool’s tokens must be the canonical wrapper (WETH9 on mainnet, the equivalent on each L2). The wrapper address is passed via the planner constructor and is the only address authorized to push raw ETH into the pool through `receive()`.

10 Security Review Process

10.1 Layered Defence

The protocol applies a defence-in-depth posture (`doc/security/threat-model.md §1`):

- **Permissioned deploy** as the primary defence against malicious tokens. `PartyPlanner.newPool` is `onlyOwner`, indefinitely. The operator is the trust anchor for token vetting.
- **Runtime guards.** `nonReentrant` on every state-mutating entry point, `killable` on every entry point except `burn`, payer-gates on the `APPROVAL` and `PREFUNDING` funding paths, deploy-time delta-equality on the initial-deposit transfer, EIP-712 witness binding on every `Permit2` path.
- **Off-chain validation.** `bin/validate-token` + `slither-check-erc` are mandatory pre-listing. The validator emits `PASS/WARN/FAIL`; `WARN` requires off-chain verification and a recorded listing-record entry, `FAIL` blocks the listing entirely.
- **Monitoring + kill switch.** Off-chain drift monitoring with documented yellow/red thresholds; `kill()` as the last-resort lever.

There is no on-chain price oracle dependency; pricing is intrinsic to the LMSR kernel.

10.2 Internal Review Artefacts

The repository carries the following internal review artefacts that gate launch (see `doc/security/`):

- `threat-model.md` — threat model, actors, fund-bearing assets, attack vectors, invariants. v2 pre-deploy.

- `checklist.md` — the security review checklist (14 sections, 84 rows). Every row resolves to either “closed in source”, “closed by docs”, or “open item”.
- `asset-authority-matrix.md` — (actor $\times$ asset $\times$ function) $\rightarrow$ gate @ file:line, the per-cell authorization matrix.
- `admin-powers.md` — CAN/CANNOT inventory of every `onlyOwner` surface, with trapped-funds analysis.
- `trusted-deployer-policy.md` — operator obligations, the token-class denylist (fee-on-transfer, rebasing, ERC-777-style hook tokens, multi-address proxy, mutable governance), the listing checklist.
- `economics-audit.md` — the economics-side review of the fee structure, single-asset round trips, and LP-favor invariants.
- `storage-layout-audit.md` — audit of the C3-linearized storage layout used under `DELEGATECALL`.
- `unchecked-blocks-audit.md` — justification for every `unchecked` block in `src/`.
- `differential-review.md` — differential review of the midpoint-*b* kernel change against the previous frozen-*b* Hanson.
- `tooling-runbook.md` — Slither, Mythril, and forge-test invocation reference.
- `user-allowance-guidance.md` — guidance for integrators on the Concierge approval surface and Permit2 usage.

10.3 Tooling

- **Foundry / forge-test.** The in-tree test suite carries the cost-preservation, midpoint-no-overshoot, imbalance-precision-sweep, and (actor $\times$ asset) gating tests that close the checklist. Coverage runs under `bin/coverage` with `forge coverage -ir-minimum -no-match-contract "ContractSize|GasTest"`.
- **Slither.** Static analysis with the project’s `slither.config.json`. False-positive suppressions are committed alongside the code they suppress and are reviewed in the checklist.
- **Token validator.** `bin/validate-token` probes a candidate token across the property axes that the protocol cannot defend against at runtime (fee-on-transfer, rebasing, hook callbacks, multi-address proxy, mutable governance). See `doc/security/trusted-deployer-policy.md`.

10.4 External Audit

The production build has been reviewed by an independent external auditor. The audit report is expected to be issued within approximately one week of this draft and will be published alongside this whitepaper at that time. [STUB: Specifics — auditor name, audit window, commit hash reviewed, report URL, and remediation summary — to be inserted on report publication.]

10.5 Bug Bounty

Liquidity Party is launching as a pre-revenue protocol and is not in a position to commit to a formal bug bounty program prior to launch. Per `SECURITY.md`, severity-graded discretionary thanks are extended for responsible reports during the interim period. The intent is to launch a formal bounty (Immunefi Boost or equivalent) once launch metrics justify the spend. **[STUB: Confirm severity bands and payout caps once committed.]**

10.6 Vulnerability Disclosure

Security reports are routed privately to `tim@dexorder.com` or via direct message to `@LiquidityParty` on X. Public GitHub issues for security vulnerabilities are explicitly discouraged. The disclosure SLA is acknowledgement within 24 hours and a coordinated disclosure timeline scaled to severity. The reporter is credited at their option once the issue is mitigated. The full policy lives in `SECURITY.md`.

10.7 Out of Scope

Per `SECURITY.md`, the following are out of scope:

- Test fixtures, mocks, and helpers under `test/`.
- Issues that require a compromised admin key as a precondition. Admin keys can `kill()` a pool and redirect protocol fees, but cannot reach LP reserves; this is documented as accepted risk in `doc/security/threat-model.md §11.N7` and `doc/security/admin-powers.md`.

11 Risk Management

11.1 Slot Drainage Is Expected LP Behaviour

Slots in a Liquidity Party pool will drain. This is normal, expected, and intentional. The protocol’s deployment strategy is to calibrate (C, κ) for maximum LP net return after impermanent loss (§9.4). Optimizing for net LP return implies that slots whose underlying asset experiences a large persistent price move *will* reach the drained state $q_k \approx 0$ before the pool internalizes the full move. This is the same trade-off every concentrated-liquidity AMM makes.

Analogy: Uniswap v3 range positions. A Uniswap v3 LP who posts liquidity over a finite price range collects fees only while the spot price is inside the range. If the price moves through the upper end of the range, the LP ends up holding 100% of the “cheaper” asset — their position has converted entirely to the appreciated asset’s pair-leg, exactly as if they had executed a market sell at every intermediate price along the range. Range LPs accept this as the fundamental staking trade-off of v3: deeper liquidity inside the range in exchange for full conversion at the boundary. Drainage in Liquidity Party is the multi-asset analogue. A pool slot is implicitly “range-staked” against the rest of the basket: while the relative price stays within the pool’s depth envelope, the slot earns fees on the LMSR quote; if the external market moves the slot’s price far enough that the pool can fund cheaper-than-external swaps until the slot empties, arbitrageurs will drain it. The maximum single-slot external move the pool can absorb before drain is

$$R_{p,\max} = \exp\left(\frac{1}{\kappa(n-1)}\right) \tag{9}$$

(§9.2); past that, the slot empties and the basket settles in the LP’s hands without that asset.

What the LP retains after a drain. A drained slot does not impair the LP’s redemption rights. Proportional `burn` continues to work and returns the LP their share of every slot that still has reserves. Single-asset `burnSwap` into the drained asset returns nothing for that slot but treats the drained slot as zero-contribution rather than reverting; `burnSwap` into a non-drained asset still works. The LP’s economic position after a slot drain is equivalent to holding the surviving basket — exactly the position a v3 LP holds after a one-sided range exit.

Industry framing. Drainage in this sense is the industry-standard cost of providing concentrated liquidity. Liquidity Party does not promise to retain every asset under every market scenario; it promises (a) pro-rata redemption of whatever the pool currently holds, (b) LP-favoring rounding and approximation at every step, and (c) a fee schedule calibrated to maximize the LP’s net return inclusive of expected drainage events. LPs participating in pools that hold uncorrelated or highly volatile assets should expect drain events on individual slots and should size their participation accordingly.

11.2 LP Loss Bound

Under constant b , classical LMSR admits a worst-case loss bound of $b \ln n$ in the payoff numéraire. With $b(\mathbf{q}) = \kappa S(\mathbf{q})$, the per-state bound scales with the current size metric: instantaneous worst-case loss $b(\mathbf{q}) \ln n = \kappa S(\mathbf{q}) \ln n$, or $\kappa \ln n$ per unit of $S(\mathbf{q})$. Because b varies with state, global worst-case loss along an arbitrary path depends on how $S(\mathbf{q})$ evolves; the proportionality clarifies how risk scales with deployed liquidity.

11.3 Liquidity-Manipulation Resistance

Every operation that changes $S(\mathbf{q})$ is priced through the same LMSR potential that prices ordinary swaps:

- Proportional mint and burn are exact scalings: $\mathbf{q} \mapsto (1 + \alpha)\mathbf{q}$ implies $b \mapsto (1 + \alpha)b$ and every pairwise marginal ratio is invariant. A proportional mint followed by its exact inverse returns the pool to the same state with no net transfer.
- Single-asset mint and burn (`swapMint`, `burnSwap`) decompose into a proportional rescaling composed with a bundle of LMSR swaps priced by the same convex potential. The cost of changing $S(\mathbf{q})$ is the sum of kernel-priced components; there is no off-market “free” lever to change b .

Because $C(\mathbf{q})$ is convex, the net kernel cost of any closed cycle of allowed operations sums to zero in the fee-free idealization; any practical implementation cost (fees, rounding, midpoint approximation error) makes the cycle strictly loss-making for the attacker. The midpoint- b approximation is bounded never to overshoot the true LS-LMSR cost, and the `swapMint`-`burnSwap` round-trip drift is bounded at ≤ 2 bps in adversarial regimes (`security/drainage-guidelines.md §7`).

11.4 Numerical Envelope

The kernel’s cost-preservation residual sits below 10^{-9} relative across the production-relevant range of pool imbalances ($R_q \leq 10^7$) and degrades gracefully past that. The ABDK `exp/ln` round-trip

noise floor is $\sim 2 \times 10^{-16}$ relative; production tolerances are bound at 10^{-9} , leaving seven orders of magnitude of headroom (see `security/drainage-guidelines.md` §3, §4).

11.5 Operational Mitigations

- User `minAmountOut` / `maxAmountIn` bounds on every state-mutating entry point.
- Capacity caps on outputs ($y \leq q_j$), enforced inside the kernel.
- Minimum-balance constraints on `initialMint` (every initial deposit must be strictly positive; $\beta_i > 0$ enforced at deploy time).
- Off-chain drift monitoring with yellow/red thresholds on the slot-drain ratio $\min q_i / (S(\mathbf{q})/n)$ and the q-ratio $\max q / \min q$.
- `kill()` as the last-resort lever, preserving proportional burn for LP exit.

11.6 Out-of-Scope Token Classes

The protocol does not support, and the trusted-deployer policy **MUST NOT** list:

- Fee-on-transfer tokens.
- Rebasing tokens.
- ERC-777-style hook tokens, ERC-677 `transferAndCall` tokens, any token that callbacks during `transfer` or `transferFrom`.
- Multi-address proxy tokens.
- Tokens whose owner can later add fees, pause transfers, or blacklist addresses (mutable post-list governance).

A token that the validator clears today but later mutates into one of the above classes is a class-2 hazard; the response is `kill()`.

12 Deployment Surface

Initial deployment. The initial production deployment targets Ethereum mainnet. The first planned expansion, if launch metrics support it, is Base. The protocol is EVM-only — non-EVM chains are out of scope for the foreseeable future. Per-chain addresses (`PartyPlanner`, `PartyInfo`, `PartyConcierge`) are published in `deployment/liqp-deployments.json` in the repository at launch. **[STUB: Mainnet addresses + deployment block + deployment date to be filled in at launch.]**

Operator key custody. The admin (planner owner / pool owner) key is held on a hardware wallet. No further detail of the operator’s internal key-management procedure is disclosed; the LP’s security guarantee does not depend on it (§2.3). The protocol is designed to be *resilient against an admin-key leak*: a leaked admin key cannot reach LP reserves under any operation in the contract surface. The worst case under a leaked key is denial-of-service (`kill()` called by the attacker) plus redirection of not-yet-collected protocol fees; LP `burn` remains operational and LPs retain their pro-rata share of reserves. The core team retains independent ability to invoke `kill()` on the planner-owned pools as a final defensive measure if the admin key is compromised.

Indexer. Liquidity Party does not host a public subgraph at this time. A canonical metadata index will be published as a static `metadata.json` artefact; the exact distribution channel is to be confirmed at launch. [STUB: Metadata index publication URL.]

Integrators that need event-level data should consume the canonical event set directly from the chain: `PartyStarted`, `Mint`, `Burn`, `Swap`, `SwapMint`, `BurnSwap`, `Flash`, `ProtocolFeesCollected`, `Killed`.

13 Disclaimers

[STUB: This section is boilerplate pending counsel review before publication.]

Issuer. Liquidity Party is operated by Dexorder Trading Services, Ltd., a company incorporated in the British Virgin Islands under registration number 2160219.

Access restrictions. The Liquidity Party front-end blocks access from addresses appearing on the U.S. Office of Foreign Assets Control (OFAC) Specially Designated Nationals (SDN) and consolidated sanctions lists. The on-chain smart contracts are permissionless to interact with from any address; the access restriction is enforced at the front-end and any sanctioned address that interacts directly with the contracts does so outside the front-end’s terms of service.

No investment advice. This whitepaper is for informational purposes only. It does not constitute investment advice, a solicitation of any security, or a representation as to the suitability of the protocol for any user, jurisdiction, or purpose. Past performance of the simulation environment described in §9.4 is not indicative of future LP returns.

No representation as to tax treatment. The tax treatment of LP participation, swap activity, flash-loan activity, and protocol-fee distribution is jurisdiction-specific and depends on facts and circumstances outside the protocol’s control. Users are responsible for their own tax compliance.

Forward-looking statements. Statements in this whitepaper regarding future deployments (Base expansion, formal bug bounty, metadata index), future audit report publication, and future protocol behaviour are forward-looking. They reflect current intent and are not commitments.

Risk acknowledgement. Liquidity provision in Liquidity Party pools is subject to impermanent loss, slot-drainage loss (§11.1), smart-contract risk, oracle-independent depegging of constituent assets, bridge risk on wrapped-bridged assets, and the residual risks documented in §11. LPs should understand these risks before depositing.

References

- Hanson, R. (2002). *Logarithmic Market Scoring Rules for Modular Combinatorial Information Aggregation*. mason.gmu.edu/~rhanson/mktscore.pdf
- Othman, A., Pennock, D., Reeves, D., Sandholm, T. (2013). *A Practical Liquidity-Sensitive Automated Market Maker*. [cs.cmu.edu/~sandholm/liquidity-sensitive market maker.EC10.pdf](http://cs.cmu.edu/~sandholm/liquidity-sensitive%20market%20maker.EC10.pdf)

- Olson, T., Claude Opus 4.7 (2026). *Loss-Versus-Rebalancing Reduced in Cross-Correlated Multi-Asset LMSR Pools*. [doc/lvr-multiasset-lmsr.pdf](#) (this repository).